Razonamiento Automático 2026/27 Challenge 3: Soluciones de entrenamiento

Arrays de bytes — la base de un mini-Tetris (Challenge de equipo)

⚠ Cómo usar este documento

Intenta primero los 21 ejercicios por tu cuenta. Compara después con esta solución **y revísala críticamente**: pueden haber soluciones distintas, pero deben hacer correctamente lo que el ejercicio pide y respetar el contrato de registros (qué recibe, qué devuelve, qué modifica).

⚡ Muy importante

Utiliza el emulador y el depurador para comprobar que tu solución hace lo que debe. Es posible hacer códigos muy diferentes, pero debes asegurar 2 cosas: 1) que son correctos, ensamblan y ejecutan y, 2) que el resultado es el que se pide, sin errores, cambios ni añadidos.

Tu mejor herramienta es el depurador del emulador: aprende a usarlo cuanto antes y ***te ahorrará horas de trabajo***, además de hacerte ***aprender muchísimo***.

ⓘ Convenio de registros de todo el Challenge

Las rutinas que devuelven un sí/no lo hacen en el **flag** (`Z` =sí, `NZ` =no), nunca en un registro — así quien llama puede saltar condicional directamente. Solo se usa un registro de retorno cuando el valor en sí tiene significado (una dirección, un valor de celda, un contador). Ninguna solución usa `push` / `pop` : cuando hacen falta dos

valores vivos a la vez, se reparten entre registros que ninguna rutina auxiliar toca (normalmente `de`).

Ejercicio 1: `board_clear`

board_clear

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Sets the whole board array to 0 (empty board)
;;
board_clear::
    ld hl, board
    ld b, BOARD_SIZE
    xor a
.do
    ld [hl+], a
    dec b
    jr nz, .do
    ret
```

Ideas clave:

- `BOARD_SIZE` (96) cabe de sobra en `b` (0-255), así que un único contador de 8 bits basta para recorrer todo el tablero.
- `xor a` deja `a=0` en una instrucción y de paso pone `Z=1`, aunque aquí no se usa el flag: es solo la forma más barata de poner un registro a 0.

Ejercicio 2: `board_init`

board_init

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Initializes everything needed to start:
;; clears the board and sets the sentinel.
;;
board_init::
    call board_clear
    ld a, BOARD_SENTINEL
    ld [board_sentinel], a
    ret
```

Ideas clave:

- `ld [board_sentinel], a` es la forma corta permitida: la restricción de "solo `a`" para direccionamiento absoluto no es un problema aquí porque el valor que queremos escribir ya está en `a`.
- El orden importa poco (podría inicializarse el centinela antes que el tablero), pero llamar a `board_clear` en vez de repetir su código refuerza la idea de reutilizar, no repetir.

> **⚡ Error típico**
> Confiar en que `board_sentinel` ya vale `$FF` al arrancar. En la Game Boy, la RAM no tiene datos iniciales garantizados: si no se escribe explícitamente por código, su valor es indeterminado.

Ejercicio 3: `board_cell_address`

board_cell_address

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Computes the memory address of a board cell,
;; given its row and column.
;; 📥 INPUT:
;;    B: row
;;    C: column
;; 🔙 RETURNS:
;;    HL: address of that cell in board
;;
board_cell_address::
    ld a, b
    add a, a
    add a, a
    add a, a
    add a, c
    ld l, a
    ld h, 0
    ld bc, board
    add hl, bc
    ret
```

Ideas clave:

- `add a,a` tres veces multiplica `a` por 2, 4 y 8 sucesivamente — mismo resultado que `fila*BOARD_WIDTH` pero sin usar multiplicación (no existe como instrucción en esta CPU).
- El índice (`fila*8+columna`) cabe siempre en un byte (máximo 95), así que `h` se puede poner a `0` directamente sin comprobar acarreo.
- `ld bc, board` sobrescribe `b` y `c` a propósito: ya no hacen falta como fila/columna, y así queda un registro de 16 bits listo para `add hl, bc` .

Ejercicio 4: `board_get_cell` / `board_set_cell`

board_get_cell / board_set_cell

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Reads the value of a board cell.
;; 📥 INPUT: B=row, C=column  🔙 RETURNS: A=value of that cell
board_get_cell::
    call board_cell_address
    ld a, [hl]
    ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Writes a value into a board cell.
;; 📥 INPUT: B=row, C=column, E=value to write
board_set_cell::
    call board_cell_address
    ld [hl], e
    ret
```

Ideas clave:

- `board_set_cell` recibe el valor en `E` y no en `A` a propósito: `board_cell_address` usa `A` internamente, así que un valor que llegara en `A` se perdería antes de poder escribirlo. `E` no lo toca, así que sobrevive a la llamada intacto.

Ejercicio 5: `board_row_is_full`

board_row_is_full

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a board row is full.
;; 📥 INPUT: B: row
;; 🔙 RETURNS (in the flag, not a register): Z=full, NZ=NOT full
;;
board_row_is_full::
    ld c, 0
    call board_cell_address
    ld b, BOARD_WIDTH
.do
    ld a, [hl]
    or a
    jr z, .not_full
    inc hl
    dec b
    jr nz, .do
    xor a
    ret
.not_full
    or 1
    ret
```

Ideas clave:

- `c=0` antes de llamar a `board_cell_address` da el inicio de la fila (columna 0); `b` (la fila, entrada de esta rutina) no se toca hasta después de la llamada, así que sigue siendo válido cuando `board_cell_address` lo necesita.
- El flag no se "deja pasar" tal cual: se fuerza explícitamente con `xor a` (todo lleno) o `or 1` (alguna vacía), en vez de confiar en el flag que deja la última comparación.

Ejercicio 6: `piece_touches_obstacle`

piece_touches_obstacle

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether the piece is at the same
;; position as the obstacle.
;; 🔙 RETURNS (in the flag, not a register): Z=match, NZ=no match
;;
piece_touches_obstacle::
    ld a, [piece_row]
    ld b, a
    ld a, [obstacle_row]
    cp b
    jr nz, .not_touching
    ld a, [piece_col]
    ld b, a
    ld a, [obstacle_col]
    cp b
    jr nz, .not_touching
    xor a
    ret
.not_touching
    or 1
    ret
```

Ideas clave:

- No hace falta ningún array ni `board_cell_address` aquí: son 4 bytes sueltos en RAM, comparados dos a dos — misma técnica que `player_x/player_y` vs `enemy_x/enemy_y` del Challenge 2.
- Si las filas ya no coinciden, no hace falta ni mirar las columnas: por eso el primer `jr nz` sale directamente.

Ejercicio 7: `board_clear_row`

board_clear_row

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Clears a whole board row.
;; 📥 INPUT: B: row
;;
board_clear_row::
    ld c, 0
    call board_cell_address
    ld b, BOARD_WIDTH
    xor a
.do
    ld [hl+], a
    dec b
    jr nz, .do
    ret
```

Ideas clave:

- Es `board_clear` (Ejercicio 1) pero acotado a una sola fila: mismo patrón de bucle, solo cambia el punto de partida (`board_cell_address` con columna 0) y el número de iteraciones (`BOARD_WIDTH` en vez de `BOARD_SIZE`).

Ejercicio 8: `board_count_filled_in_row`

board_count_filled_in_row

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Counts how many cells of a row are filled.
;; 📥 INPUT: B: row  🔙 RETURNS: A: number of filled cells (0..BOARD_WIDTH)
;;
board_count_filled_in_row::
    ld c, 0
    call board_cell_address
    ld b, BOARD_WIDTH
    ld d, 0
.do
    ld a, [hl]
    or a
    jr z, .empty
    inc d
.empty
    inc hl
    dec b
    jr nz, .do
    ld a, d
    ret
```

Ideas clave:

- `d` se usa como contador porque `board_cell_address` no lo toca (solo modifica `a`, `b`, `c`, `hl`) — sobrevive intacto durante todo el bucle.
- A diferencia de `board_row_is_full`, aquí **no** se puede salir en cuanto se encuentra una celda vacía: hay que recorrerlas todas para contarlas.

> ⚡ **Error típico**
> Olvidar el `inc hl` del bucle (o ponerlo en el sitio equivocado). Sin avanzar `hl`, la rutina comprueba la misma celda una y otra vez.

Ejercicio 9: `board_copy_row_down`

board_copy_row_down

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Copies a row into the row immediately
;; below it (row -> row+1).
;; 📥 INPUT: B: source row
;;
board_copy_row_down::
    ld c, 0
    call board_cell_address   ;; hl = dirección de (fila, 0): origen
    ld d, h
    ld e, l                   ;; de = origen (copia de hl)
    ld bc, BOARD_WIDTH
    add hl, bc                ;; hl = origen + BOARD_WIDTH = destino (fila+1, 0)
    ld b, BOARD_WIDTH
.do
    ld a, [de]
    ld [hl+], a
    inc de
    dec b
    jr nz, .do
    ret
```

Ideas clave:

- La dirección de la fila siguiente **no** necesita una segunda llamada a `board_cell_address`: es la del origen `+ BOARD_WIDTH`, así que basta un `add hl, bc`.
- Como no hay `push`/`pop` en este Challenge, guardar el origen en `de` (copiando `hl` byte a byte con `ld d,h` / `ld e,l`) es lo que permite tener origen y destino vivos a la vez.

> ⚡ **Error típico (tentador, pero inviable sin `push`/`pop`)**
> Llamar dos veces a `board_cell_address` (una para el origen, otra para `fila+1`). Parece la vía "más directa", pero solo `d`/`e` sobreviven a una llamada, y

hacen falta a la vez la dirección de 16 bits del origen **y** el valor `fila+1` — no caben ambos sin guardar algo en la pila.

Ejercicio 10: `board_column_has_block_above`

board_column_has_block_above

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether any cell is filled in a
;; column, looking from row 0 up to the given
;; row (not included).
;; 📥 INPUT: B: column, D: number of rows to check (from row 0)
;; 🔙 RETURNS (in the flag, not a register): NZ=found, Z=none
;;
board_column_has_block_above::
    ld c, b         ;; c = columna (la entrada llegó en b)
    ld b, 0         ;; b = fila = 0 (punto de partida)
    call board_cell_address   ;; hl = dirección de (0, columna)
    ld bc, BOARD_WIDTH
.do
    ld a, [hl]
    or a
    jr nz, .found
    add hl, bc
    dec d
    jr nz, .do
    xor a
    ret
.found
    or 1
    ret
```

Ideas clave:

- La entrada llega al revés del contrato de `board_cell_address` (`B=fila,C=columna`): hay que mover la columna a `C` y poner `B=0` **antes** de llamarla.

- `d` (número de filas a comprobar) no lo toca `board_cell_address`, así que se puede decrementar directamente como contador del bucle, sin copiarlo a otro sitio.
- La polaridad aquí es `NZ` =encontrado, al revés que el resto del Challenge: es la que sale "gratis" de la comparación, sin forzar nada — fíjate siempre en el contrato de cada rutina en vez de asumir la convención general.

Ejercicio 11: `piece_move_left` / `piece_move_right`

piece_move_left / piece_move_right

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Decrements the column pointed to by HL,
;; without going below 0.
;; 📥 INPUT: HL = address of the column to move (e.g. &piece_col)
piece_move_left::
    ld a, [hl]
    or a
    ret z
    dec [hl]
    ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Increments the column pointed to by HL,
;; without exceeding BOARD_WIDTH-1.
;; 📥 INPUT: HL = address of the column to move (e.g. &piece_col)
piece_move_right::
    ld a, [hl]
    cp BOARD_WIDTH - 1
    ret z
    inc [hl]
    ret
```

Ideas clave:

- `dec [hl]` / `inc [hl]` modifican directamente el byte en memoria, sin pasar por `a`: no hace falta leer, modificar y volver a escribir a mano.

- El límite derecho es `BOARD_WIDTH-1` (7), **no** `BOARD_WIDTH` (8): comparar contra 8 dejaría "colarse" una columna de más.

Ejercicio 12: `piece_move_from_joypad`

piece_move_from_joypad

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Reads the D-Pad and moves piece_col by
;; calling piece_move_left/piece_move_right as needed.
;;
piece_move_from_joypad::
    ld hl, $FF00
    ld a, %00100000   ;; Select Direction keys group
    ld [hl], a
    ld a, [hl]
    ld a, [hl]
    ld a, [hl]        ;; stabilize: read 3 times
    ld b, a           ;; save the stable reading in B (not A)
    ld hl, piece_col
    bit I_RIGHT_BIT, b
    jr nz, .no_right
    call piece_move_right
.no_right
    bit I_LEFT_BIT, b
    jr nz, .no_left
    call piece_move_left
.no_left
    ret
```

Ideas clave:

- El grupo de teclas se selecciona igual que en el Challenge 2 (indirecto por `hl`, porque `$FF00` no admite otro registro que no sea `a` en direccionamiento absoluto), pero con el valor que selecciona **direcciones** (`%00100000`) en vez de botones.
- La lectura se guarda en `b`, no en `a`, precisamente porque `piece_move_left` / `piece_move_right` sí modifican `a` internamente: si la lectura se quedara en `a`, se perdería al llamar a la primera de las dos.

- Ninguna de las dos rutinas toca `hl`, así que no hace falta recargarlo entre la comprobación de `I_RIGHT_BIT` y la de `I_LEFT_BIT`: sigue apuntando a `piece_col`.

Ejercicio 13: `board_copy_row_to_tilemap`

board_copy_row_to_tilemap

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Copies a row of BOARD_WIDTH cells from the
;; board to the tilemap.
;; 📥 INPUT: DE=source in board, HL=destination in tilemap
;; 🔙 On return: DE and HL advanced BOARD_WIDTH each
;; ❌ Does not use C
;;
board_copy_row_to_tilemap::
    ld b, BOARD_WIDTH
.do
    ld a, [de]
    ld [hl+], a
    inc de
    dec b
    jr nz, .do
    ret
```

Ideas clave:

- El contrato "no toca `C`" no es decorativo: los Ejercicios 14 y 15 necesitan `C` libre para su propia aritmética de salto de fila mientras llaman a esta rutina una vez por fila.
- Solo funciona correctamente si se ejecuta en VBLANK/HBLANK o con la pantalla apagada (escritura a VRAM) — en este Challenge se asume que sí, pero en hardware real haría falta esperar a VBLANK antes de llamarla.

Ejercicio 14: `board_render_to_tilemap`

board_render_to_tilemap

```
DEF TILEMAP_1_START = $9800
DEF TILEMAP_WIDTH   = 32
DEF NEXT_ROW_OFF    = TILEMAP_WIDTH - BOARD_WIDTH

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Walks the whole board row by row and copies
;; it to the tilemap, calling
;; board_copy_row_to_tilemap for each one.
;;
board_render_to_tilemap::
    ld de, board
    ld hl, TILEMAP_1_START
.do
    call board_copy_row_to_tilemap
    ld bc, NEXT_ROW_OFF
    add hl, bc
    ld a, e
    cp LOW(board_end)
    jr nz, .do
    ld a, d
    cp HIGH(board_end)
    jr nz, .do
    ret
```

Ideas clave:

- Terminar comparando `de` contra `board_end` (la etiqueta de "Código común", justo después de la última celda) evita llevar un contador de filas aparte, y deja `bc` completamente libre para el salto de fila del tilemap.
- La comparación de un puntero de 16 bits se hace **byte a byte**: primero la parte baja (`LOW(board_end)`), y solo si coincide, la parte alta (`HIGH(board_end)`) — no hay ninguna instrucción que compare 16 bits de golpe.
- No se usa `DEF BOARD_END = board + BOARD_SIZE` : como la `SECTION "Board", WRAM0` de "Código común" no tiene dirección fija, esa expresión no es una constante en tiempo de ensamblado (el enlazador decide dónde va `board` después). Funcionaría si la `SECTION` tuviera una dirección fijada (Ej. `SECTION`

`"Board", WRAM0[$C100]`). Una etiqueta (`board_end:`) sí funciona siempre, tenga o no dirección fija la sección.

Ejercicio 15: `board_render_to_tilemap_sentinel`

board_render_to_tilemap_sentinel

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Walks the whole board row by row and copies
;; it to the tilemap, stopping when it finds
;; BOARD_SENTINEL instead of comparing pointers.
;;
board_render_to_tilemap_sentinel::
    ld de, board
    ld hl, TILEMAP_1_START
.do
    ld a, [de]
    cp BOARD_SENTINEL
    ret z
    call board_copy_row_to_tilemap
    ld bc, NEXT_ROW_OFF
    add hl, bc
    jr .do
```

Ideas clave:

- El centinela vive justo después de la última celda (`board_sentinel`, colocado ahí por el propio `SECTION` de "Código común"), así que cuando `de` llega ahí, leer `[de]` da `BOARD_SENTINEL` en vez de un valor de celda real.
- Misma tarea que el Ejercicio 14, con menos código (no necesita `board_end` ni `LOW()` / `HIGH()`) a cambio de "gastar" un valor reservado (`$FF`) que no se puede usar como tile/color real.

Ejercicio 16: `board_piece_fits_bounds`

board_piece_fits_bounds

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a rectangular piece fits
;; within the board's bounds (without checking
;; for blocks).
;; 📥 INPUT: B: origin row, C: origin column, D: piece width, E: piece height
;; 🔙 RETURNS (in the flag, not a register): Z=fits, NZ=goes outside
;;
board_piece_fits_bounds::
    ld a, c
    add a, d
    cp BOARD_WIDTH + 1
    jr nc, .out
    ld a, b
    add a, e
    cp BOARD_HEIGHT + 1
    jr nc, .out
    xor a
    ret
.out
    or 1
    ret
```

Ideas clave:

- `columna+ancho <= BOARD_WIDTH` es lo mismo que `columna+ancho < BOARD_WIDTH+1` (son enteros): por eso se compara contra `BOARD_WIDTH+1` y se salta con `nc` (equivalente a "mayor o igual").
- El salto a `.out` **no** hace `ret nc` directamente: fuerza el flag con `or 1` en su propio bloque. Si se devolviera el flag "tal cual" deja la resta, el caso en que la pieza se pasa por exactamente 1 (`columna+ancho == BOARD_WIDTH+1`) pondría `Z` y `NC` a la vez, y se devolvería el flag equivocado.

Ejercicio 17: board_check_z_piece_fits

board_check_z_piece_fits

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a Z-piece fits: inside the
;; board and its 4 cells empty.
;;
;;   Piece shape (3x2 bounding box):
;;     X X .
;;     . X X
;;
;; 📥 INPUT: B: origin row (top-left corner of the 3x2 box), C: origin column
;; 🔙 RETURNS (in the flag, not a register): Z=fits, NZ=does NOT fit
;;
board_check_z_piece_fits::
    ld d, 3
    ld e, 2
    call board_piece_fits_bounds
    ret nz
    call board_cell_address     ;; hl = celda 1: (fila, columna)
    ld a, [hl]
    or a
    jr nz, .not_fit
    inc hl                      ;; celda 2: (fila, columna+1)
    ld a, [hl]
    or a
    jr nz, .not_fit
    ld bc, BOARD_WIDTH
    add hl, bc                  ;; celda 3: (fila+1, columna+1)
    ld a, [hl]
    or a
    jr nz, .not_fit
    inc hl                      ;; celda 4: (fila+1, columna+2)
    ld a, [hl]
    or a
    jr nz, .not_fit
```

```
    xor a
    ret
.not_fit
    or 1
    ret
```

Ideas clave:

- `board_piece_fits_bounds` no modifica `b` ni `c` (solo los lee): tras la llamada siguen siendo fila/columna válidos para pasárselos a `board_cell_address`.
- `board_piece_fits_bounds` ya devuelve en el mismo convenio de flags que necesita esta rutina (`Z` =cabe, `NZ` =no cabe), así que `ret nz` es un `return` directo, sin traducir nada.
- Solo hace falta **una** llamada a `board_cell_address`: el resto de direcciones son la primera más un desplazamiento fijo (+1 para el lado, `+BOARD_WIDTH` para bajar una fila) — intentar una segunda llamada perdería la primera dirección, igual que en el Ejercicio 9.

Ejercicio 18: `piece_fits_bounds_check`

piece_fits_bounds_check

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a piece fits by size, reading
;; its X,Y,W,H directly from the piece data.
;; 📥 INPUT: HL: address of the piece (starts at X)
;; 🔙 RETURNS (in the flag, not a register): Z=fits by size, NZ=does NOT fit
;;
piece_fits_bounds_check::
    ld a, [hl+]
    ld c, a         ;; c = X = columna
    ld a, [hl+]
    ld b, a         ;; b = Y = fila
    ld a, [hl+]
    ld d, a         ;; d = W = ancho
    ld a, [hl+]
    ld e, a         ;; e = H = alto
    jp board_piece_fits_bounds
```

Ideas clave:

- `ld a, [hl+]` cuatro veces seguidas lee `X,Y,W,H` en orden y avanza `hl` solo, sin necesitar índices ni direccionamiento adicional.
- El final es un salto (`jp`), no una llamada (`call`): como no queda nada por hacer después, `board_piece_fits_bounds` puede devolver directamente a quien llamó a `piece_fits_bounds_check` — se ahorra un `ret` y sigue siendo correcto.

Ejercicio 19: `hl_move_by_increment`

hl_move_by_increment

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves HL one step in the direction given
;; by an increment.
;; 📥 INPUT: HL: current position, A: increment (%1111DULR, negative logic)
;; 🔙 RETURNS: HL: moved one step in the given direction
;; ❌ Modifies BC (used to shift by a row)
;;
hl_move_by_increment::
    bit I_RIGHT_BIT, a
    jr nz, .no_right
    inc hl
.no_right
    bit I_LEFT_BIT, a
    jr nz, .no_left
    dec hl
.no_left
    bit I_DOWN_BIT, a
    jr nz, .no_down
    ld bc, BOARD_WIDTH
    add hl, bc
.no_down
    bit I_UP_BIT, a
    jr nz, .no_up
    ld bc, -BOARD_WIDTH
    add hl, bc
.no_up
    ret
```

Ideas clave:

- Como cada incremento tiene **un solo** bit a 0 (lógica negativa), las 4 comprobaciones se pueden encadenar sin ningún `if-else`: solo una de las cuatro llegará a ejecutar su `inc` / `dec` / `add`.
- No existe `sub hl, rr` en esta CPU: moverse "hacia arriba" es sumar el negativo de `BOARD_WIDTH` (`ld bc, -BOARD_WIDTH` + `add hl, bc`), no restar.

Ejercicio 20: piece_fits_no_bounds

piece_fits_no_bounds

```
;;;
```

Ideas clave:

- Hacen falta dos punteros vivos a la vez (uno para recorrer la pieza, otro para el tablero): `de` guarda el de la pieza porque ni `board_cell_address` ni `hl_move_by_increment` lo tocan.
- El bucle comprueba primero la celda **actual**, y solo después decide si hay que seguir: el centinela (`0`) se lee, pero no se usa como incremento — simplemente indica que ya no quedan más celdas que comprobar.

Ejercicio 21: `piece_draw_no_bounds`

piece_draw_no_bounds

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Draws a piece on the board (writes 1 into
;; each of its cells). Assumes it already fits.
;; 📥 INPUT: HL: address of the piece (starts at X)
;;
piece_draw_no_bounds::
    ld d, h
    ld e, l
    ld a, [de]
    ld c, a
    inc de
    ld a, [de]
    ld b, a
    inc de
    inc de
    inc de
    call board_cell_address
.do
    ld a, 1
    ld [hl], a
    ld a, [de]
    inc de
    or a
    ret z
    call hl_move_by_increment
    jr .do
```

Ideas clave:

- Mismo recorrido que el Ejercicio 20 (mismo montaje de `b` / `c` / `de`), cambiando solo qué se hace con cada celda: en vez de comprobar, escribe `1`.
- El orden es "dibuja, luego decide si sigue": la última celda se dibuja igual que las demás, y el centinela solo sirve para saber cuándo parar el bucle.